# ScannerBase — Plan de pruebas y resultados QA

Nov 5, 2026 · @En Una Regeneración Facial, Que Es Mejor, Primero Poner Bio Est · SecDevOps Banco Base v0.8

## Resumen ejecutivo

Se encontraron 58 hallazgos en 26 módulos (~20 500 líneas) y se corrigieron 56; los 2 restantes requieren una decisión tuya o un backend. Siete eran críticos: varios hacían que una etapa completa no funcionara o exponían secretos.

| Indicador | Antes | Después |
|---|---|---|
| Pruebas unitarias e integración (61) | 28 pasan · 33 fallan | 61 pasan |
| Pruebas funcionales end-to-end en la app real (21) | falla en F-04, F-08 y se cae | 21 pasan |
| Pruebas de estrés (13) | 2 fallan (congelamiento 12,6 s y 27 s) | 13 pasan |
| Resoluciones y escalas probadas (8) | 3 con defectos visuales | 0 defectos |
| Casos de fuzzing de parsers y especificaciones (3 100) | 1 158 caídas | 0 caídas |
| Excepciones de Tk o hilos en todas las corridas | — | 0 |

| Severidad | Hallazgos | Corregidos |
|---|---|---|
| Crítica | 7 | 7 |
| Alta | 18 | 17 (SEC-03 necesita backend) |
| Media | 22 | 22 |
| Baja | 9 | 8 (OBS-01 por confirmar) |
| Mejoras agregadas | 2 | 2 |

Los siete críticos, en una línea cada uno:

- **BUG-33** — El DAST/API con ZAP + Nuclei (motor por defecto sin licencia Probely) fallaba siempre: importaba el módulo equivocado.
- **BUG-31** — El estado «acumulado» era global: el reporte, gate, riesgo e inventario de la app B incluían hallazgos de la app A.
- **BUG-20** — Crear una app con el mismo nombre que otra (o con otras mayúsculas) sobrescribía su perfil e historial.
- **BUG-14** — La tarea de Windows heredaba «solo con corriente»: en laptops con batería los escaneos programados nunca corrían.
- **BUG-08** — Las escrituras JSON no eran atómicas: un cierre o lectura concurrente podía perder inventario, horarios o configuración.
- **SEC-06** — En cadenas de conexión, la contraseña quedaba en claro en secrets.json, el reporte y el CSV.
- **SEC-08** — snyk-win.exe se descargaba y ejecutaba sin hash y, si fallaba TLS, sin verificar certificado.

El detalle de los 58 hallazgos y de cada caso de prueba está en la matriz Excel adjunta.

## Alcance, entorno y estándares

Se probó todo el código fuente de ScannerBase v0.8 (zip del 5-nov-2026) con sus imágenes, en Linux con pantalla virtual (Xvfb) y Python 3.12 + Tk 8.6. La plataforma soportada es solo Windows 10/11; lo exclusivo de Windows se simuló o se dejó para prueba manual.

| Elemento | Detalle |
|---|---|
| Código bajo prueba | 26 módulos, ~20 500 líneas: GUI Tkinter, pipeline SCA/SAST/Secretos/Contenedores/IaC/DAST/API, Probely, ServiceNow, Programador de tareas, política de administración |
| Entorno | Linux + Xvfb, Python 3.12.3, Tk 8.6, sin acceso a Snyk, Probely, ServiceNow, Adoptium ni Maven (red restringida) |
| Servicios externos | Simulados con dobles de prueba que devuelven salidas reales de cada herramienta |
| Resoluciones | 1024×768, 1280×720, 1366×768, 1920×1080, 3840×2160 |
| Escalas DPI | 100 %, 125 %, 150 %, 200 % (nueva variable BBSCANNER_UI_SCALE) |
| Temas | Claro y oscuro en cada pestaña |

Estándares aplicados:

- **ISO/IEC/IEEE 29119-3** — estructura del plan, casos con ID, pasos, resultado esperado y obtenido, trazabilidad hallazgo → caso.
- **ISO/IEC 25010** — características evaluadas: adecuación funcional, fiabilidad, eficiencia de desempeño, usabilidad, seguridad, mantenibilidad y portabilidad.
- **OWASP ASVS 4.0 y CWE Top 25** — revisión de seguridad del propio escáner: manejo de secretos, integridad de descargas, TLS, inyección (CSV, encoded query, href).
- **Pruebas de regresión antes/después** — cada suite corrió contra el código original y el corregido para demostrar cada defecto y su corrección.

## Plan de pruebas por nivel

El plan tiene siete niveles; cada uno tiene su criterio de salida y todos se cumplieron con el código corregido.

| Nivel | Qué cubre | Técnica | Criterio de salida |
|---|---|---|---|
| 1. Análisis estático | Compilación, nombres indefinidos, referencias módulo.atributo rotas, código muerto, SAST del propio código | py_compile, ruff (F821 con `import *` resuelto), bandit, verificador AST propio | 0 errores; hallazgos de bandit revisados uno por uno |
| 2. Unitarias e integración | Hallazgos/CWE/OWASP, SARIF, persistencia JSON, política de admin, inventario, horarios, tarea de Windows, ServiceNow, reportes HTML/CSV, motor de secretos, parsers | pytest con dobles de prueba; casos límite, datos corruptos, rutas maliciosas | 100 % de casos pasan |
| 3. Fuzzing | Salidas de ZAP, Nuclei, Trivy, Checkov; OpenAPI, Swagger 2 y Postman | 3 100 variantes con tipos inesperados, nulos y listas vacías | 0 caídas: una entrada mala no tumba la etapa |
| 4. Funcional end-to-end | La app real: alta de apps, escaneo real de secretos, artefactos, aislamiento entre apps, bloqueo de cambio de app, cancelación, horarios, 8 diálogos, atajos, persistencia, cierre | Automatización de ScannerApp en Xvfb | 21/21 sin excepciones de Tk o de hilos |
| 5. Interfaz y resoluciones | Cada pestaña en claro y oscuro, asistente de primer uso, menú lateral, barra de estado | Capturas a 5 resoluciones y 4 escalas, revisión visual | Nada cortado ni fuera de pantalla |
| 6. Estrés y rendimiento | 500 apps, 200 horarios, 12 000 hallazgos en pantalla, 100 000 líneas de consola, 40 cambios de tema, 160 diálogos, 400 escrituras concurrentes | Medición de tiempos y memoria (psutil), perfilado (cProfile) | UI responde en menos de 3 s por acción, sin fuga creciente ni corrupción |

| Nivel | Qué cubre | Técnica | Criterio de salida |
|---|---|---|---|
| 7. Seguridad del escáner | Exposición de secretos, integridad de descargas, TLS, XSS en reporte, inyección CSV y de consultas, escalamiento a admin, path traversal | Revisión de código guiada por ASVS + pruebas dirigidas | Sin secretos en claro en ninguna salida; sin ejecutables sin verificar |

## Resultados por nivel

Con el código corregido todos los niveles pasan; las mediciones de rendimiento clave están abajo.

| Nivel | Resultado final | Lo más relevante que encontró |
|---|---|---|
| Análisis estático | 0 errores | 12 llamadas a funciones inexistentes en oss_web_scanner (BUG-33); 2 módulos duplicados sin uso |
| Unitarias e integración | 61/61 (antes 28/61) | Escritura JSON no atómica, CWE inventados, SARIF inválido, contraseña en claro, apps sobrescritas |
| Fuzzing | 0/3 100 caídas (antes 1 158) | Una entrada mal formada descartaba todos los hallazgos válidos de la etapa |
| Funcional end-to-end | 21/21 | Cambio de app a mitad de escaneo; reporte de una app con hallazgos de otra |
| Interfaz y resoluciones | 8/8 configuraciones limpias | Menú cortado a 720–768 px de alto; ventana más ancha que la pantalla a 1024 px; tarjetas y enlaces cortados |
| Estrés y rendimiento | 13/13 | Congelamiento de 12,6 s y 27 s con miles de secretos |
| Seguridad | 9 hallazgos, 8 corregidos | Ejecutable sin verificar, token Snyk por TLS sin validar, XSS en enlaces del reporte, inyección CSV |

| Medición de estrés | Antes | Después |
|---|---|---|
| Mostrar 12 000 hallazgos en la pestaña Código | 12,6 s | 1,1 s |
| Cambiar de tema con esos hallazgos cargados | 27,4 s | 0,9 s |
| Arranque con 500 apps y 200 horarios | 2,2 s | 2,2 s |
| Refrescar y buscar en 500 apps | 0,22 s / 0,34 s | igual |
| 40 cambios de tema (reconstruye toda la UI) | +25 MB y luego estable (+0 MB en 10 más) | igual, sin fuga |
| Abrir y cerrar diálogos 40 veces (Esc) | 0 MB, 0 widgets, 0 hooks retenidos | igual, sin fuga |
| 400 escrituras concurrentes al inventario | — | 0 errores, 0 archivos corruptos |
| Reporte HTML con 12 000 hallazgos | 1,2 s, 17 MB | igual |
| Motor de secretos: 4 000 archivos y 800 000 líneas | 8,2 s | 7,8 s |

La consola retiene como máximo 20 000 líneas; 100 000 líneas seguidas tardan 17,7 s en mostrarse y suman solo 12 MB de memoria.

## Hallazgos críticos y altos

Los 25 hallazgos críticos y altos están corregidos salvo SEC-03, que necesita un backend. Los medios y bajos están en la matriz Excel, pestaña «Hallazgos».

| ID | Módulo | Severidad | Hallazgo | Corrección |
|---|---|---|---|---|
| BUG-08 | `ui_common.write_json` | Crítica | Escritura no atómica: una lectura concurrente veía JSON a medias y `read_json` devolvía vacío; un cierre o corte de luz a mitad de escritura dejaba inventario, horarios o configuración corruptos, y se perdían en silencio | Temporal + fsync + `os.replace`, lock por archivo y reintento si el antivirus bloquea |

| ID | Módulo | Severidad | Hallazgo | Corrección |
|---|---|---|---|---|
| BUG-14 | Programador de Windows | Crítica | La tarea se creaba con «schtasks /SC MINUTE», que hereda «Iniciar solo con corriente alterna»: en laptops con batería los escaneos con la app cerrada nunca corrían | Registro por XML (batería permitida, StartWhenAvailable, IgnoreNew, límite 12 h); las tareas antiguas se re-registran solas |
| BUG-20 | `AppInventoryStore.save_app` | Crítica | Crear una app con el mismo nombre que otra (o que solo difiere en mayúsculas o símbolos) sobrescribía su perfil e historial sin aviso | Id único garantizado para apps nuevas |
| SEC-06 | `secrets_scanner` (contexto) | Crítica | En cadenas de conexión con varios valores, cada hallazgo ocultaba solo el suyo: el hallazgo del usuario mostraba la contraseña en claro en secrets.json, el reporte HTML y el CSV | Se ocultan todos los valores detectados en la línea |
| SEC-08 | `static_scanner._install_standalone` | Crítica | snyk-win.exe, que luego se ejecuta, se descargaba sin verificar hash y, si fallaba TLS, sin verificar certificado: ejecución de código vía proxy o MITM | SHA-256 publicado obligatorio, sin reintento inseguro, escritura atómica |
| BUG-31 | Estado acumulado ( `ScanStateStore` ) | Crítica | El estado «acumulado» era global, no por app: un DAST de la app B incluía en reporte, gate, riesgo e inventario los hallazgos SCA/SAST/Secretos de la app A | Estado por aplicación, en la carpeta de la app |
| BUG-33 | `oss_web_scanner` | Crítica | `import report_engine as fz` en lugar de `findings` : todo DAST/API con ZAP + Nuclei (motor por defecto sin Probely) fallaba al procesar resultados | Corregido y verificado con un chequeo automático de referencias módulo.atributo en todo el proyecto |
| BUG-05 | `findings.iac_key` | Alta | Una línea con rango («12-14») o lista lanzaba ValueError y abortaba la fusión de hallazgos IaC | Corregido |
| BUG-09 | `ui_common.read_json` | Alta | Un JSON corrupto se descartaba sin respaldo y la siguiente escritura borraba los datos | Respaldo «.corrupt-fecha» |
| BUG-12 | `admin_policy.load` | Alta | Sin `admin_policy.json` , devolvía diccionarios compartidos con los valores por defecto: cualquier cambio en memoria los contaminaba | Copia profunda |
| SEC-01 | `admin_policy.is_admin` | Alta | `BBSCANNER_ADMIN=1` daba acceso de administrador, y a la clave de Probely, también en el .exe distribuido | Solo funciona con el código fuente |
| BUG-15 | Programador de Windows | Alta | /TR admite 261 caracteres: rutas largas (OneDrive, carpetas con nombre completo) hacían fallar el registro | El XML no tiene ese límite |

| ID | Módulo | Severidad | Hallazgo | Corrección |
|---|---|---|---|---|
| BUG-19 | `_load_app_profile` | Alta | Se podía cambiar de app durante un escaneo (doble clic en inventario, «Cargar», asistente): objetivos y resultados quedaban en el perfil equivocado | Bloqueo mientras hay escaneo |
| BUG-23 | Editor de aplicación | Alta | Guardar el editor abierto durante un escaneo revertía conteos, riesgo e historial del escaneo recién terminado | Se fusiona con la versión en disco |
| BUG-24 | `_persist` tras horario | Alta | Tras un escaneo programado se restauraba un perfil viejo en memoria; editar cualquier campo revertía el resultado | Se guarda sobre la versión en disco |
| BUG-25 | ServiceNow (registro de tickets) | Alta | `servicenow_tickets.json` truncado se reemplazaba por uno vacío: se perdía el seguimiento (tickets duplicados o que no se cierran) | Respaldo + escritura atómica |
| SEC-04 | `report_engine` (HTML) | Alta | URLs de hallazgos DAST/API iban sin filtrar en href: un «javascript:» del sitio escaneado se ejecutaba al hacer clic en el reporte | Filtro `safe_url` en los 9 enlaces + rel="noopener noreferrer" |
| SEC-05 | `report_engine.export_csv` | Alta | Inyección de fórmulas: títulos o evidencias que empiezan con = + - @ se ejecutaban al abrir el CSV en Excel | Se neutralizan con apóstrofo |
| BUG-29 | `secrets_scanner` | Alta | Líneas de más de 4 000 caracteres se omitían: llaves de API en JS minificado o JSON de una línea no se detectaban | Ventanas solapadas |
| SEC-09 | `static_scanner._api_get` | Alta | El token de Snyk viajaba por TLS sin verificar si la conexión verificada fallaba | Reintento inseguro solo con `BBSCANNER_INSECURE_TLS=1` explícito |
| BUG-32 | Historial de remediación | Alta | Comparaba contra el último escaneo de cualquier app: escanear la app B marcaba como remediados los hallazgos de la A; el archivo crecía sin límite | Historial por app, tope de 200 escaneos y 5 000 acciones |
| BUG-34 | Parsers ZAP/Nuclei/Trivy/Checkov | Alta | Una entrada mal formada abortaba toda la etapa (959 caídas en 1 600 casos de fuzzing) | Tolerancia por entrada: 0 caídas |
| BUG-37 | Pestaña Código (Secretos) | Alta | 2 000 tarjetas = 18 000 widgets: 12,6 s al mostrar resultados y 27 s al cambiar de tema | Paginación de 150 + «Mostrar más» (1,1 s y 0,9 s) |
| BUG-38 | Arranque ( `ui_common` ) | Alta | Con la carpeta del programa sin permiso de escritura (Program Files, red de solo lectura) moría al importar, sin ventana ni mensaje | data, Reports y tools pasan a `%LOCALAPPDATA%\SecDevOpsBancoBase` |

| ID | Módulo | Severidad | Hallazgo | Corrección |
|---|---|---|---|---|
| SEC-03 | `test_servicenow_ticket` ( `_SVC_BLOB` ) | Alta (riesgo aceptado) | La contraseña de la cuenta de integración de producción va ofuscada en el programa con una llave derivable del propio código | Sin corrección posible sin backend: mover los tickets al backend FastAPI planeado y rotar la credencial |

## Lo que no se pudo probar aquí: para hacerlo tú

Estas pruebas necesitan Windows real, credenciales o red corporativa; las marcadas **Prioridad 1** tocan código que cambié y que solo pude validar con simulaciones.

### Prioridad 1 — cambios míos que dependen de Windows o de la red

- [ ] Programador de tareas (BUG-14/15): activa «Usar el Programador de tareas de Windows» en Automatizar. En `taskschd.msc` abre la tarea `SecDevOpsBancoBase_ScanScheduler` y confirma que en Condiciones está desmarcado «Iniciar solo si el equipo está conectado a la corriente». Luego crea un horario, cierra la app, desconecta el cargador y verifica que el escaneo corre.
- [ ] Programador sin permisos de administrador: repite lo anterior con un usuario estándar; si falla, confirma que aparece el aviso y que los horarios siguen corriendo con la app abierta.
- [ ] Programador con ruta larga: instala la app en una ruta de más de 261 caracteres (por ejemplo dentro de OneDrive) y activa la opción; antes fallaba.
- [ ] Mover la carpeta del programa y reabrir: la tarea debe repararse sola («Tarea registrada y apuntando a la ubicación actual»). Una tarea creada por la versión anterior debe re-registrarse una vez.
- [ ] Descarga de snyk-win.exe (SEC-08): usa «Snyk → Reinstalar (limpio)». Debe decir «SHA-256 verificado ✓». Si el proxy corporativo rompe TLS, ahora se rechaza: instala la CA corporativa en Windows o define `NODE_EXTRA_CA_CERTS` con la ruta del .pem.
- [ ] Lista de organizaciones de Snyk detrás del proxy (SEC-09): si ya no carga, prueba primero con la CA; usa `BBSCANNER_INSECURE_TLS=1` solo temporalmente.
- [ ] DAST real con ZAP + Nuclei (BUG-33) contra un sitio de QA, perfil Rápido y Estándar; confirma que el reporte trae hallazgos de ambos motores. Antes esta etapa fallaba siempre.
- [ ] API real con una especificación OpenAPI y una colección Postman de un proyecto del banco.

- [ ] Arranque desde `C:\Program Files` (BUG-38): la app debe abrir y guardar data, Reports y tools en `%LOCALAPPDATA%\SecDevOpsBancoBase`. Confirma que esa ubicación te parece bien.
- [ ] Reconstruir el .exe con PyInstaller: eliminé `core/scheduler_os.py` y `core/servicenow_tickets.py` (no se usaban); quítalos del .spec si los listaba. Prueba `ScannerBase.exe --tick` y `--force`.

**Prioridad 2 — integraciones con servicios reales**

- [ ] Snyk: inicio de sesión SSO, SCA y SAST sobre un repositorio real, con una y con varias carpetas.
- [ ] git-secrets con Git Bash en un repositorio git (BUG-30): el mismo secreto debe salir una sola vez y marcado como validado por dos motores.
- [ ] ServiceNow dev con Modo prueba apagado: crear, actualizar y cerrar un ticket, y adjuntar el reporte. Prueba un nombre de aplicación con «^» (SEC-02).
- [ ] Probely: app asignada, validación de clave y respaldo a ZAP + Nuclei cuando Probely falla.
- [ ] Contenedores con Trivy (imagen de registro, .tar y SBOM) e IaC con Checkov, Trivy y SonarQube local: descarga de motores, JRE y bases de datos, incluido el espejo interno de Administración.
- [ ] Grabador de login (Selenium + pynput) en Chrome y Edge, con exclusión de logout.
- [ ] Instalación de Python vía winget desde el .exe (Checkov) y reparación de tcl/tk en un Python sin ese componente.

**Prioridad 3 — experiencia en Windows**

- [ ] Escalas reales de 125 %, 150 % y 175 %, y monitor 4K. Aquí usé fuentes de reemplazo porque Linux no trae Segoe UI ni Segoe Fluent Icons: revisa íconos y anchos de texto.
- [ ] Dos monitores: abre la app en el secundario, ciérrala, desconéctalo y vuelve a abrir (BUG-03): debe aparecer en el principal.
- [ ] Cierre durante un escaneo: debe pedir confirmación (UX-01). Mata el proceso desde el Administrador de tareas a mitad de un escaneo y confirma que al reabrir ofrece restaurarlo.
- [ ] Cierre repetido tras varios escaneos: vigila el error «Tcl_AsyncDelete» que reportaste antes; no se reproduce en Linux.
- [ ] CSV en Excel: acentos correctos (BUG-26) y una evidencia que empiece con «=» debe verse como texto (SEC-05).
- [ ] Reporte HTML en Edge y Chrome; eliminar un reporte manda la carpeta a la Papelera.

- [ ] `admin_policy.json` editado en Bloc de notas (con BOM) y distribuido en una carpeta de solo lectura.

## Riesgos residuales, recomendaciones y re-ejecución

El mayor riesgo pendiente es SEC-03: la credencial de producción de ServiceNow viaja dentro del programa.

- **SEC-03** — Mueve la creación de tickets al backend FastAPI que ya planeas para el sitio y rota la contraseña de `Integration_user`; mientras tanto, mantén esa cuenta con permisos mínimos.
- **OBS-01** — Si no se detecta la organización de Snyk, el ticket de producción se envía con «TEST-ORG». Confirma si es intencional.
- **Cambio de comportamiento** — El estado acumulado y el historial de remediación ahora viven en la carpeta de cada app. El primer escaneo de cada app tras actualizar no mostrará resultados anteriores de otras etapas; los archivos globales viejos se conservan y «Remediación» los abre si la app aún no tiene historial.
- **Recomendación** — Agrega las pruebas entregadas a tu flujo antes de cada versión: habrían detectado BUG-33 en segundos.

Para re-ejecutar todo (Python 3.12 con tkinter; en Windows sin Xvfb):

1. `pip install pytest psutil jinja2 pyyaml requests send2trash pillow selenium`
2. Pruebas unitarias, de integración y fuzzing: `SRC=<carpeta del código> python -m pytest pruebas/tests`
3. Funcionales en la app real: `SRC=<carpeta> bash pruebas/harness/runfunc.sh func`
4. Recorrido visual por resolución: `bash pruebas/harness/runtour.sh r1366 1366x768`
5. Estrés: `APP_DIR=<copia> python pruebas/harness/stress.py`
6. Referencias rotas: `python pruebas/harness/check_attrs.py <carpeta del código>`

Los scripts de las secciones 3 a 5 asumen Linux con Xvfb; en Windows corre `functional.py`, `tour.py` y `stress.py` directamente con `APP_DIR` apuntando a una copia del programa.